

AEGISFIN-AI

Complete End-to-End Project Blueprint

An explainable, policy-governed financial intelligence system combining predictive ML, fraud detection, regulatory RAG, agentic reasoning, human oversight, evaluation and auditable decisioning.

Document scope	What this blueprint covers
Product vision	Problem, users, use cases, success criteria and non-goals
AI/ML system	Credit risk, fraud detection, explainability and model evaluation
GenAI system	Regulatory RAG, grounding, prompts, citations and evaluation
Agentic system	Supervisor/orchestrator, specialist agents, tool use and guardrails
Decision governance	Policy engine, human-in-the-loop, approval gates and audit trail
Engineering	FastAPI, database, frontend, services, security, testing and deployment
Execution roadmap	Phase-by-phase implementation, acceptance criteria and deliverables

Working principle

AI recommends. Deterministic policy constrains. Humans control high-impact decisions. Evidence and actions are auditable.

Prepared as the master build document for the AegisFin-AI portfolio project.

Table of Contents

Section	Topic
1	Executive Summary
2	Problem Statement and Product Vision
3	Core Use Cases and User Journeys
4	Target Architecture
5	Technology Stack
6	Repository and Service Structure
7	Phase 0 - Requirements, Scope and Design Contracts
8	Phase 1 - Credit Risk ML
9	Phase 2 - Fraud Detection
10	Phase 3 - Explainability and Risk Evidence
11	Phase 4 - Regulatory RAG
12	Phase 5 - Agentic Decision Layer
13	Phase 6 - Policy Engine and Human Oversight
14	Phase 7 - Audit, Governance and Safety
15	Phase 8 - Backend API and Database
16	Phase 9 - Frontend / Operations Dashboard
17	Phase 10 - Evaluation and Benchmarking
18	Phase 11 - Testing and Reliability
19	Phase 12 - Security and Privacy
20	Phase 13 - Deployment and MLOps
21	Phase 14 - Demo, Documentation and Resume Packaging
22	End-to-End Data and Decision Flow
23	Key APIs and Data Contracts
24	Evaluation Metrics and Acceptance Criteria
25	What Must Not Be Added
26	Final Definition of Done
27	Resume / Interview Story

1. Executive Summary

AegisFin-AI is a portfolio-grade financial intelligence platform. It is not a single model and it is not an LLM wrapper. The system combines predictive machine learning, fraud analytics, explainable AI, retrieval-augmented generation over financial regulations/policies, controlled AI agents, deterministic decision policies, human review and append-only audit records.

The central design objective is to make financial AI useful without making the language model the source of authority. Models estimate risk, retrieval supplies evidence, agents organize reasoning, policy code constrains actions, and human reviewers handle cases that cross predefined thresholds. This architecture is deliberately similar in spirit to high-trust financial software: probabilistic components are separated from deterministic controls.

Primary outcome: a recruiter or interviewer should be able to follow one financial case from raw input -> model predictions -> explanations -> regulatory evidence -> agent recommendation -> policy decision -> human approval (when required) -> audit record.

Portfolio positioning

- Flagship project demonstrating ML + GenAI + agentic AI + responsible AI + backend engineering.
- Focused on explainable and auditable decision support rather than autonomous money movement or autonomous lending approval.
- Designed to be evaluated quantitatively, not just demonstrated visually.

Important scope boundary

AegisFin-AI is a decision-support and risk-intelligence platform for a controlled sandbox. It must not be presented as production banking software, an actual credit approval authority, or a guarantee of financial outcomes. Real-world financial deployment would require institution-specific policies, compliance review, data governance, security controls and regulatory approval.

2. Problem Statement and Product Vision

Financial institutions generate several partially disconnected signals: credit behavior, transaction activity, fraud indicators, customer attributes and policy/regulatory constraints. A model can produce a score, but a practical analyst also needs to understand why the score exists, which policies apply, what evidence supports a recommendation and whether a human must intervene.

AegisFin-AI unifies those steps into one workflow. It should answer four questions for every case:

- What is the risk? Predictive models estimate credit and fraud risk.
- Why is the risk present? Explainability surfaces important features and evidence.
- What rules apply? Regulatory/policy retrieval grounds the reasoning in source material.
- What should happen next? An agent proposes a bounded action that is checked by deterministic policy and human-review rules.

Product principles

Principle	Implementation rule
Deterministic where possible	Use ordinary code for thresholds, eligibility, routing, retry limits, approval gates and arithmetic.
Grounded generative AI	LLMs must cite retrieved regulatory/policy evidence and clearly separate evidence from interpretation.
No unrestricted autonomy	Agents cannot directly change credit limits, approve money movement or bypass policy gates.
Explainability by default	Every model result has feature-level evidence and model/version metadata.
Auditability	Persist decision inputs, outputs, evidence references, policy results, user actions and timestamps.
Evaluation before polish	A dashboard is secondary to measurable model/RAG/agent performance.

3. Core Use Cases and User Journeys

ID	Use case	Journey
UC-01	Credit risk assessment	Upload/enter applicant profile -> score probability of default -> show risk band + SHAP explanations -> retrieve applicable policy -> produce recommendation -> route to human review if threshold crossed.
UC-02	Fraud investigation	Receive transaction -> fraud model + anomaly checks -> enrich with customer/transaction context -> explain suspicious features -> retrieve investigation policy -> agent summarizes evidence -> investigator reviews.
UC-03	Regulatory question	Analyst asks a policy/regulation question -> retrieve relevant chunks -> LLM answers only from evidence -> show citations and source metadata -> allow feedback.
UC-04	Case review	Reviewer opens a case -> sees model scores, explanations, retrieved evidence, agent recommendation and policy decision -> approves/rejects/escalates -> system records action.
UC-05	Model evaluation	Admin runs a fixed dataset -> stores metrics, confusion matrix/PR/ROC, calibration and subgroup slices -> compares against baseline -> records model version.

Primary personas

Persona	Needs
Risk Analyst	Understand credit/fraud signals, compare cases, inspect explanations.
Compliance Analyst	Ask policy questions, inspect citations, verify that recommendations respect policy.
Reviewer / Approver	Make or confirm the final high-impact decision.
Model / Platform Engineer	Train, evaluate, version and monitor models and retrieval pipelines.
Admin	Manage users, thresholds, policies, datasets and system health.

4. Target Architecture

The system should be implemented as a modular full-stack application with clear boundaries between probabilistic models, retrieval/LLM components and deterministic controls.

Logical architecture



Recommended flow

Input case -> validate -> feature build -> ML scoring -> explanations -> retrieval -> agent synthesis -> policy evaluation -> approval gate -> persist decision -> audit -> dashboard

Architecture rule

The agent layer must call services through explicit tools/functions. The model never receives direct database write privileges. All privileged actions are mediated by APIs and policy checks.

5. Technology Stack

Layer	Recommended technology	Purpose
Language	Python 3.x	ML, APIs, agents, evaluation
API	FastAPI + Pydantic	Typed REST API, validation, OpenAPI
ML	pandas, NumPy, scikit-learn, LightGBM	Training and risk/fraud models
Explainability	SHAP	Feature-level explanations
GenAI	OpenAI-compatible LLM or Groq-compatible provider	Regulatory reasoning / agent synthesis
RAG	LangChain or lightweight custom pipeline + vector DB	Chunking, retrieval, citations
Embeddings	SentenceTransformers / embedding API	Semantic retrieval
Vector store	FAISS for local or Chroma/Pinecone for service mode	Regulatory document index
Database	PostgreSQL (production), SQLite (local MVP)	Users, cases, scores, policies, audit records
Frontend	React + TypeScript + Vite	Operations/reviewer dashboard
Charts	Recharts / similar	Metrics and case visualizations
Auth	JWT/OAuth-compatible layer	Identity and role checks
Testing	pytest + httpx + frontend tests	Unit/API/integration tests
MLOps	MLflow (recommended)	Experiment/model registry and versions
Packaging	Docker / Docker Compose	Reproducible local stack
Deployment	Render / Railway / AWS / similar	Demo deployment
Observability	Structured logs + metrics	Latency/errors/model-call tracing

Do not adopt every library simultaneously. The implementation should start with a small stable stack and add infrastructure only when a requirement justifies it.

6. Repository and Service Structure

A monorepo is recommended because the project is a single portfolio system and cross-service contracts should remain easy to inspect.

```
AegisFin-AI/
├── backend/
│   ├── app/
│   │   └── main.py
│   ├── core/ # config, security, logging
│   ├── api/ # routers / HTTP endpoints
│   ├── schemas/ # Pydantic request/response models
│   ├── models/ # ORM / persistence models
│   ├── services/ # business services
│   ├── policies/ # deterministic policy engine
│   ├── agents/ # orchestrator + specialist agents
│   ├── rag/ # ingestion, retrieval, citations
│   ├── ml/ # inference wrappers
│   ├── audit/ # audit writer / event schemas
│   └── tests/
├── ml/
│   ├── credit_risk/
│   ├── fraud/
│   ├── evaluation/
│   ├── artifacts/
│   └── rag/
│       ├── documents/
│       ├── ingestion/
│       ├── indexes/
│       └── eval/
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   ├── hooks/
│   │   ├── types/
│   │   └── tests/
│   ├── scripts/
│   ├── docs/
│   ├── data/ # ignored raw data / samples
│   ├── docker-compose.yml
│   ├── .env.example
│   ├── requirements.txt / pyproject.toml
│   └── README.md
```

Repository rule

Every phase should leave the repository in a runnable state. Avoid one giant final merge. Each phase gets tests, documentation and a small demo path.

Phase 0 - Requirements, Scope and Design Contracts

Objective: Freeze the product contract before building model/agent complexity.

Inputs / dependencies

- Define target users and three core workflows: credit case, fraud case, regulatory question.
- Define the exact meaning of score, recommendation, review and audit event.
- Choose local-demo datasets and synthetic records where public data cannot be used.

Implementation

- Write product requirements document and architecture decision record.
- Define JSON schemas for Case, RiskScore, Evidence, Recommendation, PolicyDecision and AuditEvent.
- Define hard safety boundaries: no autonomous monetary transfer, no direct DB writes from agents, human review for high-impact cases.
- Define acceptance thresholds before implementation.

Main artifacts / files

```
docs/requirements.md
docs/architecture.md
backend/app/schemas/*
docs/policies.md
```

Tests / validation

- Schema validation tests.
- Contract examples for each endpoint.
- Threat-model review of privileged actions.

Exit criteria

- A new contributor can understand the system in 15 minutes.
- All core objects have stable typed contracts.
- Safety boundaries are explicit.

Phase 1 - Credit Risk ML

Objective: Build a rigorous, explainable probability-of-default style classifier as the first production-like AI component.

Inputs / dependencies

- Existing credit-risk dataset and preprocessing pipeline.
- Train/validation/test split strategy that avoids leakage.
- Class-imbalance strategy and metric definitions.

Implementation

- Perform EDA: missingness, distributions, imbalance, cardinality and leakage checks.
- Feature engineering with reproducible transformers.
- Train a strong tabular baseline and LightGBM model.
- Tune with cross-validation using PR-AUC/ROC-AUC as appropriate.
- Calibrate probabilities and choose risk bands with documented thresholds.
- Version preprocessing, model, feature list and metrics.

Main artifacts / files

```
ml/credit_risk/train.py
ml/credit_risk/preprocess.py
ml/credit_risk/predict.py
ml/evaluation/credit_metrics.py
ml/artifacts/model_version.json
```

Tests / validation

- ROC-AUC, PR-AUC, Gini, KS.
- Calibration curve / Brier score where applicable.
- Confusion matrix for thresholded bands.
- Leakage checks and reproducible rerun.

Exit criteria

- Model beats the chosen baseline on predefined metrics.
- Same input produces deterministic inference artifact.
- Prediction includes model version and risk band.

Phase 2 - Fraud Detection

Objective: Add a transaction-risk model that complements the credit model and creates a second independent ML signal.

Inputs / dependencies

- Transaction dataset or carefully designed synthetic transaction generator.
- Fraud label definition and time-aware split.
- Features covering amount, velocity, device/customer context and temporal behavior.

Implementation

- Start with a simple baseline.
- Add LightGBM/XGBoost-style gradient boosting or another tabular model.
- Engineer velocity and behavioral features without leakage.
- Evaluate rare-event performance using PR-AUC and recall at fixed precision.
- Create a fraud score with interpretable risk bands.
- Keep fraud inference independent from the language model.

Main artifacts / files

```
ml/fraud/train.py
ml/fraud/features.py
ml/fraud/predict.py
ml/evaluation/fraud_metrics.py
data/synthetic/generate_transactions.py
```

Tests / validation

- Time-aware validation.
- Precision/recall/F1 and PR-AUC.
- Recall at operational precision targets.
- False-positive review sample.

Exit criteria

- Fraud model has documented baseline comparison.
- Feature pipeline is reproducible.
- Inference output is consumable by the case API.

Phase 3 - Explainability and Risk Evidence

Objective: Turn model outputs into evidence a reviewer can understand and challenge.

Inputs / dependencies

- Trained credit and fraud models.
- SHAP-compatible inference path.

Implementation

- Generate global feature importance and per-case SHAP explanations.
- Map technical features to reviewer-friendly labels.
- Create reason codes / top contributing factors.
- Store explanation model version with every case.
- Never let a generated explanation invent a reason that is absent from model evidence.

Main artifacts / files

```
ml/explainability/shap_service.py
backend/app/services/explanation_service.py
backend/app/schemas/evidence.py
```

Tests / validation

- Explanation consistency tests.
- Top-feature values match input case.
- Missing/invalid feature handling.
- Golden cases with expected reason codes.

Exit criteria

- Every score has top factors, direction, contribution and model version.
- Reviewer can reproduce the explanation from the same case.

Phase 4 - Regulatory RAG

Objective: Ground AI reasoning in a curated, versioned corpus of regulations and internal policy documents.

Inputs / dependencies

- Authoritative PDF/HTML/text sources selected for the demo.
- Chunking and metadata strategy.
- Embedding model and vector store.

Implementation

- Ingest documents with source, title, version/date and page/section metadata.
- Normalize text and chunk without losing section context.
- Create embeddings and store metadata with each chunk.
- Retrieve top-k candidates and optionally rerank.
- Generate answers that cite source chunks.
- Add no-answer behavior when evidence is insufficient.
- Keep source documents and indexes versioned.

Main artifacts / files

```
rag/ingestion/parse.py
rag/ingestion/chunk.py
rag/retrieval/search.py
rag/generation/answer.py
rag/eval/dataset.jsonl
```

Tests / validation

- Recall@k / hit rate.
- Citation correctness.
- Groundedness / faithfulness review.
- Answer refusal when the corpus lacks evidence.

Exit criteria

- Every regulatory answer contains source references.
- Unanswerable questions do not receive fabricated policy claims.
- Document/index versions are visible.

Phase 5 - Agentic Decision Layer

Objective: Create a controlled orchestration layer that combines model scores, evidence and policy tools into a structured recommendation.

Inputs / dependencies

- Credit/fraud inference services.
- RAG retrieval tool.
- Case data API.
- Typed recommendation schema.

Implementation

- Use a supervisor/orchestrator rather than free-form multi-agent chat.
- Define specialist tools/agents: Risk Analyst, Fraud Analyst, Compliance Researcher, Decision Synthesizer.
- Use structured outputs only.
- Require citations for compliance claims.
- Give agents read-only tools by default.
- Return recommendation + rationale + evidence IDs + uncertainty + required review flag.

Main artifacts / files

```
backend/app/agents/orchestrator.py
backend/app/agents/risk_agent.py
backend/app/agents/fraud_agent.py
backend/app/agents/compliance_agent.py
backend/app/agents/decision_agent.py
```

Tests / validation

- Mock-tool tests.
- Schema-valid structured output tests.
- Prompt injection / malicious document tests.
- Missing-tool / timeout / invalid-output fallbacks.

Exit criteria

- Agent never returns an unstructured final action.
- Every recommendation maps to evidence and policy inputs.
- Agent failure degrades safely to human review.

Phase 6 - Policy Engine and Human Oversight

Objective: Separate AI recommendation from final authority using deterministic policy rules and approval gates.

Inputs / dependencies

- Risk scores, model reasons, agent recommendation, user role.
- Approved policy table / thresholds.

Implementation

- Implement explicit rules for approve/review/reject or equivalent bounded outcomes.
- Add threshold bands and exception rules.
- Add role-based approval permissions.
- Require human review for high-impact or conflicting signals.
- Implement kill-switch / fail-safe mode for agent or model outages.
- Make policy decisions deterministic and testable without an LLM.

Main artifacts / files

```
backend/app/policies/engine.py
backend/app/policies/rules.py
backend/app/services/review_service.py
backend/app/schemas/policy.py
```

Tests / validation

- Rule unit tests.
- Boundary-value tests.
- Conflicting-signal tests.
- Unauthorized approval tests.
- Fail-closed tests for missing evidence.

Exit criteria

- Same inputs always produce same policy result.
- Agents cannot bypass policy.
- High-impact cases are routed to an authorized human.

Phase 7 - Audit, Governance and Safety

Objective: Make every material decision traceable and reviewable.

Inputs / dependencies

- Case lifecycle and policy events.
- User identity and role.
- Model / prompt / index version metadata.

Implementation

- Write append-only audit events for case creation, scoring, retrieval, recommendation, policy decision, review and final status.
- Store correlation IDs linking all events in one case.
- Record timestamps, actor, source versions and decision reason.
- Add audit export in JSON/CSV.
- Add retention and redaction policy for sensitive data.

Main artifacts / files

```
backend/app/audit/event_writer.py
backend/app/models/audit.py
backend/app/api/audit.py
docs/audit-schema.md
```

Tests / validation

- Event immutability test.
- Correlation ID continuity.
- Missing audit event detection in integration tests.
- Export/replay test.

Exit criteria

- A reviewer can reconstruct how the final decision was produced.
- No privileged action occurs without an audit record.

Phase 8 - Backend API and Database

Objective: Expose the complete decision workflow through a clean typed backend.

Inputs / dependencies

- Defined schemas and services from previous phases.
- Database choice and migration system.

Implementation

- Build FastAPI app with routers by domain.
- Add validation and error handling.
- Persist users, cases, scores, evidence, documents, recommendations, policy decisions and audit events.
- Add pagination, filtering and case status APIs.
- Add health/readiness endpoints.
- Generate OpenAPI documentation.

Main artifacts / files

```
backend/app/main.py
backend/app/api/*.py
backend/app/models/*.py
backend/app/schemas/*.py
backend/app/services/*.py
backend/tests/
```

Tests / validation

- Unit tests for services.
- API tests with httpx/TestClient.
- Database migration tests.
- Authorization tests.
- Error-contract tests.

Exit criteria

- All critical workflows work through API without direct script calls.
- OpenAPI documents the public contract.
- Database state is consistent with audit events.

Phase 9 - Frontend / Operations Dashboard

Objective: Provide a polished reviewer interface that makes the system explainable rather than merely decorative.

Inputs / dependencies

- Stable API and data contracts.
- Case/recommendation/audit endpoints.

Implementation

- Dashboard cards: cases, high-risk, pending review, model metrics.
- Case explorer with search/filter.
- Case detail: risk scores, SHAP reasons, transaction anomalies, regulatory evidence, agent recommendation and policy result.
- Reviewer action panel with role-aware buttons.
- Audit timeline.
- Model evaluation page.
- RAG query page with citations.

Main artifacts / files

```
frontend/src/pages/Dashboard.tsx
frontend/src/pages/CaseDetail.tsx
frontend/src/pages/ReviewQueue.tsx
frontend/src/pages/Evaluation.tsx
frontend/src/pages/RegulatorySearch.tsx
```

Tests / validation

- Loading/error/empty-state tests.
- Role-based UI tests.
- Case rendering against API fixtures.
- Manual accessibility and responsive review.

Exit criteria

- A reviewer can complete an end-to-end case without developer tools.
- Every visual claim links back to an API field or evidence record.

Phase 10 - Evaluation and Benchmarking

Objective: Quantitatively prove that each AI component adds value.

Inputs / dependencies

- Fixed datasets and golden test cases.
- Baseline rules / simple models.

Implementation

- Credit benchmark against baseline model.
- Fraud benchmark against baseline.
- RAG benchmark with labeled question-answer-source set.
- Agent benchmark with structured-case scenarios.
- Policy benchmark for deterministic correctness.
- Measure latency, cost and failure rates.
- Generate one reproducible evaluation report and save artifacts.

Main artifacts / files

```
ml/evaluation/run_all.py
ml/evaluation/reports/
rag/eval/metrics.py
backend/tests/e2e/
```

Tests / validation

- Credit: ROC-AUC, PR-AUC, Gini, KS, calibration.
- Fraud: PR-AUC, precision, recall, F1, recall at fixed precision.
- RAG: hit rate/Recall@k, citation correctness, groundedness, no-answer rate.
- Agent: schema validity, tool success, evidence coverage, safe fallback rate.
- System: p50/p95 latency, error rate, cost per case.

Exit criteria

- One command reproduces evaluation.
- Metrics have documented dataset version and seed.
- Weaknesses are visible instead of hidden.

Phase 11 - Testing and Reliability

Objective: Make failures deterministic, diagnosable and safe.

Inputs / dependencies

- All services available in test mode.
- Fixtures and synthetic datasets.

Implementation

- Unit-test pure functions first.
- Add integration tests across ML/RAG/policy boundaries.
- Add end-to-end tests for three critical workflows.
- Add property/boundary tests for policy thresholds.
- Test provider outages, malformed LLM outputs, empty retrieval, DB failures and duplicate requests.

Main artifacts / files

```
backend/tests/unit/  
backend/tests/integration/  
backend/tests/e2e/  
frontend/src/**/*.test.*
```

Tests / validation

- Minimum target: critical backend services >90% statement coverage.
- 100% coverage for policy rules and safety gates.
- All end-to-end golden workflows pass in CI.

Exit criteria

- CI blocks merges on critical regressions.
- Failures surface actionable logs and correlation IDs.

Phase 12 - Security and Privacy

Objective: Treat financial-style data as sensitive even in a student project.

Inputs / dependencies

- Threat model.
- Auth/role design.
- Secret management strategy.

Implementation

- Never commit API keys.
- Hash passwords; never store raw credentials.
- Use JWT/session controls with expiry and role claims.
- Validate uploads and restrict document types.
- Sanitize logs and redact sensitive fields.
- Rate-limit expensive LLM endpoints.
- Validate all tool arguments.
- Use allowlists for external network access where practical.

Main artifacts / files

```
backend/app/core/security.py
backend/app/core/config.py
docs/security.md
.env.example
```

Tests / validation

- Auth bypass tests.
- IDOR tests.
- Prompt injection tests.
- Secret scanning.
- Upload abuse tests.
- Rate-limit test.

Exit criteria

- No secrets in repository.
- Privileged endpoints require role.
- Sensitive information is absent from ordinary logs.

Phase 13 - Deployment and MLOps

Objective: Turn the local project into a reproducible online demo without pretending it is a regulated production deployment.

Inputs / dependencies

- Dockerized backend/frontend.
- Environment variables.
- Model and vector artifacts.

Implementation

- Create Dockerfiles and docker-compose for local demo.
- Configure managed PostgreSQL/vector store if needed.
- Add health checks.
- Deploy backend and frontend.
- Store model/version manifests.
- Add experiment tracking.
- Document rollback to previous model version.

Main artifacts / files

```
docker-compose.yml
backend/Dockerfile
frontend/Dockerfile
mlflow/ or experiment tracking config
docs/deployment.md
```

Tests / validation

- Fresh-machine setup test.
- Health endpoint monitoring.
- Smoke test after deployment.
- Model rollback test in staging.

Exit criteria

- A clean environment can reproduce the project.
- Public demo works without exposing secrets.
- Versions are traceable.

Phase 14 - Demo, Documentation and Resume Packaging

Objective: Package the project so the work is legible to recruiters and interviewers.

Inputs / dependencies

- Working system and evaluation report.
- Architecture diagram.

Implementation

- Write README with architecture, setup, screenshots, evaluation and tradeoffs.
- Record one 3-5 minute demo path.
- Prepare architecture and model cards.
- Prepare resume bullets based only on measured facts.
- Add limitations and future work.
- Create a final project report/presentation.

Main artifacts / files

```
README.md
docs/model-card-credit.md
docs/model-card-fraud.md
docs/agent-card.md
docs/evaluation-report.md
docs/demo-script.md
```

Tests / validation

- Fresh clone README test.
- Demo rehearsal with no hidden manual steps.
- All benchmark numbers trace to saved artifacts.

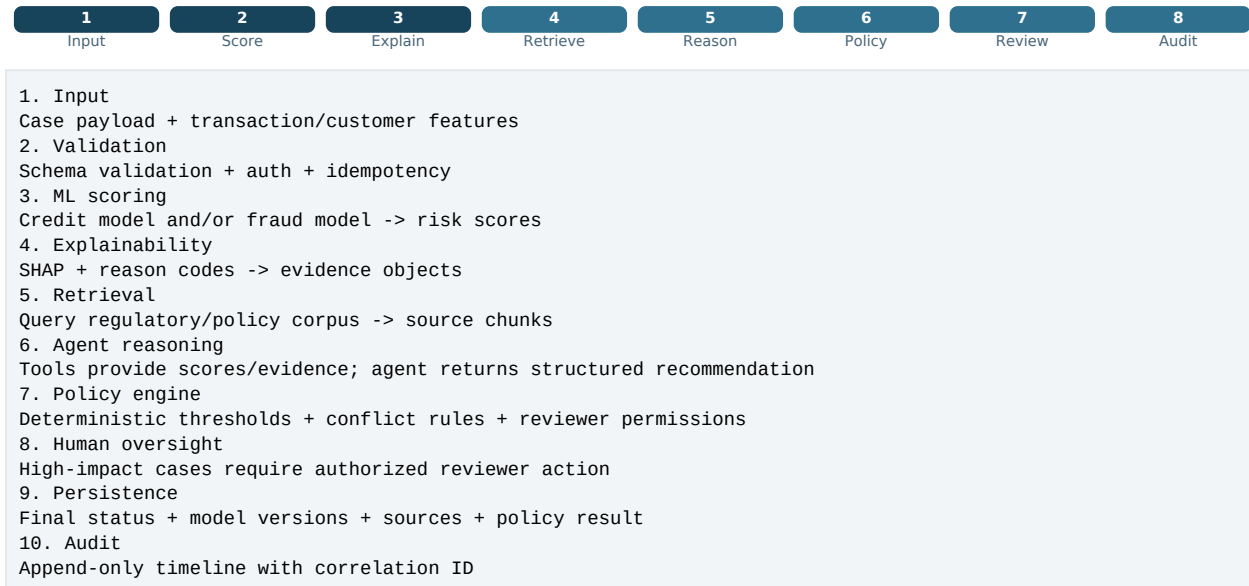
Exit criteria

- Someone unfamiliar with the project can run it and understand why each component exists.

22. End-to-End Data and Decision Flow

The system should be explainable at the level of individual data transformations. The complete flow below is the reference implementation path.

Execution sequence



Golden case example

A user submits a credit case with moderate predicted default risk but multiple fraud indicators. The credit model produces a score and SHAP factors. The fraud model produces a high anomaly score. The compliance retriever finds a policy clause requiring manual review when fraud indicators exceed a threshold. The agent summarizes the evidence and recommends MANUAL_REVIEW. The policy engine independently verifies that the thresholds are crossed, and the case enters the reviewer queue. When the reviewer acts, the action and policy version are appended to the audit record.

23. Key APIs and Data Contracts

The backend contract should be understandable even without reading implementation details. Endpoint names can evolve, but domain boundaries should remain stable.

Method	Endpoint	Purpose
POST	/auth/login	Authenticate user and issue session/JWT
POST	/cases	Create a new decision-support case
GET	/cases	List/filter cases
GET	/cases/{id}	Case summary
POST	/cases/{id}/score	Run credit/fraud scoring
GET	/cases/{id}/explanations	Retrieve SHAP/reason codes
POST	/rag/query	Regulatory/policy question with citations
POST	/cases/{id}/recommend	Run controlled agent recommendation
GET	/cases/{id}/policy	Return deterministic policy result
POST	/cases/{id}/review	Authorized human action
GET	/cases/{id}/audit	Case audit timeline
POST	/evaluations/run	Run benchmark batch
GET	/evaluations/{id}	Evaluation metrics and artifact links
GET	/health	Liveness
GET	/ready	Readiness

Core case schema

Case

- case_id
- customer_ref
- case_type: CREDIT | FRAUD | COMBINED
- input_features
- transaction_context (optional)
- status
- created_at
- created_by

RiskScore

- score
- risk_band
- model_name
- model_version
- threshold_version
- generated_at

Evidence

- evidence_id
- type: SHAP | REGULATION | TRANSACTION | POLICY
- source_ref
- excerpt_or_feature
- value
- contribution

Recommendation

- action
- confidence / uncertainty
- rationale
- evidence_ids[]
- required_review
- agent_version

PolicyDecision

- decision
- reasons[]
- policy_version
- permitted_actions[]

AuditEvent

- event_id
- case_id
- actor
- event_type
- payload_hash / summary
- timestamp
- correlation_id

24. Evaluation Metrics and Acceptance Criteria

The final report should contain both model performance and operational/system quality. A project is not considered complete because a classifier has a high ROC-AUC or an LLM produces persuasive prose.

Component	Primary metrics	What success means
Credit model	ROC-AUC, PR-AUC, Gini, KS, calibration	Strong separation with calibrated, reproducible outputs and no leakage
Fraud model	PR-AUC, precision, recall, F1, recall at fixed precision	Useful rare-event detection with an understood false-positive tradeoff
SHAP	Explanation correctness/consistency	Reasons match input and model behavior
RAG	Recall@k, citation correctness, groundedness, no-answer rate	Answers are supported by retrieved sources and abstain when evidence is absent
Agent	Schema validity, evidence coverage, safe fallback, tool success	Structured recommendation uses tools and fails safely
Policy engine	100% critical rule test pass	Deterministic and fail-closed on missing/invalid evidence
Backend	p50/p95 latency, error rate	Stable service behavior under demo workload
Audit	Event completeness, replayability	A reviewer can reconstruct the case timeline
Security	Auth/authorization test pass, secret scan	No obvious privilege bypass or secret leakage

Baseline comparison

- Credit: compare against a simple baseline before tuning LightGBM.
- Fraud: compare against a simple baseline and show the operating point.
- RAG: compare retrieval against keyword-only or naive retrieval if practical.
- Agent: compare controlled tool-based agent against unconstrained free-form output only as a safety/format demonstration, not as a performance race.
- System: report cost and latency per case so LLM usage is not hidden.

25. What Must Not Be Added

Avoid features that sound impressive but reduce the credibility of the system. The project should remain a coherent financial decision-support platform.

- Do not let an LLM directly approve/deny a high-impact financial action without a deterministic policy and human review path.
- Do not add a multi-agent swarm where a single orchestrator plus a few specialist tools is sufficient.
- Do not train an ML model on labels that were generated by your own deterministic rules unless the experiment explicitly studies imitation or policy learning.
- Do not claim regulatory compliance because a RAG pipeline quotes regulations. The system only demonstrates policy-aware reasoning.
- Do not report inflated accuracy on imbalanced fraud data. Show precision, recall, PR-AUC and operational tradeoffs.
- Do not use synthetic data as if it were representative of a real bank. Label it clearly and explain limitations.
- Do not store API keys, raw credentials or unnecessary personal data in the repository.
- Do not make the dashboard the centerpiece. Evaluation, architecture and traceability are the centerpiece.

This project should look like an engineer built a controlled decision system, not like an LLM was placed at the center of a banking application.

26. Final Definition of Done

AegisFin-AI is complete when all of the following are true:

- A user can create and inspect a complete credit/fraud case through the frontend and API.
- Credit risk inference is reproducible, versioned and evaluated against a baseline.
- Fraud inference is reproducible, versioned and evaluated against a baseline.
- Each score includes explanation evidence and model/version metadata.
- Regulatory RAG returns grounded answers with source citations and abstains when evidence is insufficient.
- The agent layer returns structured recommendations and can only access explicitly defined tools.
- A deterministic policy engine independently determines what actions are permitted.
- High-impact or conflicting cases are routed to an authorized human reviewer.
- Every material step produces an auditable event linked by correlation ID.
- An evaluation batch reproduces stored metrics and artifacts.
- Critical policy/safety logic has comprehensive automated tests.
- The application can be launched from a clean environment using documented steps.
- No secrets are committed and sensitive fields are redacted from ordinary logs.
- The README explains architecture, setup, limitations, evaluation and demo flow.
- The final resume claims use only measured results that can be demonstrated.

Recommended final demo sequence

```
00:00 Show architecture and the core design principle.
00:30 Create or open a credit/fraud case.
01:00 Show ML scores + SHAP explanations.
01:40 Ask a regulatory question and inspect citations.
02:10 Run the controlled agent recommendation.
02:40 Show deterministic policy result and required review.
03:10 Approve/review the case as an authorized user.
03:40 Open the audit timeline and reconstruct the decision.
04:15 Open evaluation page and show benchmark results.
04:45 Show repository structure / deployment health.
05:00 Close with limitations and future work.
```

27. Resume / Interview Story

Recommended project title: AegisFin-AI - Explainable, Policy-Governed Financial Intelligence Platform

Resume framing

- Built an end-to-end financial risk intelligence platform combining tabular ML for credit/fraud risk, SHAP-based explanations, regulatory RAG and tool-driven agentic decision support.
- Designed a policy-governed architecture where AI produces recommendations while deterministic rules and human approval gates control high-impact decisions; recorded decisions in an auditable case timeline.
- Implemented reproducible evaluation for risk models, retrieval quality, agent reliability and end-to-end system performance, including baseline comparisons and versioned artifacts.

Interview questions this project should prepare you to answer

- Why did you use LightGBM for tabular financial risk data?
- Why is PR-AUC important for fraud compared with accuracy?
- How did you prevent data leakage?
- What does SHAP actually tell the reviewer?
- Why does the LLM need RAG?
- How do you verify a regulatory answer is grounded?
- Why should the agent not be the final authority?
- How does the policy engine differ from the agent?
- What happens when the LLM provider is unavailable?
- How would you monitor drift in production?
- What are the privacy and governance limitations?
- Which components are deterministic and which are probabilistic, and why?

The strongest answer to all of these is visible in the architecture: probabilistic components estimate and explain; retrieval supplies evidence; agents synthesize; deterministic policies constrain; humans handle high-impact decisions; audit records preserve traceability.

Final build priority

Priority	Focus
P0	Requirements/contracts, credit model, fraud model, explanations, policy engine, API, tests
P1	RAG, agent orchestration, audit trail, reviewer dashboard
P2	MLflow/model registry, observability, deployment polish, advanced evaluation
P3	Optional enhancements only after the core workflow is stable

Source / reference note

This blueprint is a project execution document based on the AegisFin-AI concept and implementation direction established for this project. Current external references consulted include the project repository context and the Razorpay Buildathon comparison used to identify useful architecture patterns. Technology/library choices should be pinned and verified against their official documentation at implementation time.